이력서(황혁주)_2026



👤 프로필 요약

타겟팅 조건이 얼마나 복잡해도 serving은 0.11ms입니다. 300만 유저 × AND/OR/NOT 중첩 조건을 배치가 흡수하고 단일 PK 조회로 처리하는 구조를 광고 플랫폼 초기부터 직접 설계한 4년 차 백엔드 개발자입니다. DAU 100만 e-commerce와 광고 플랫폼 환경에서 복잡한 요구사항을 확장 가능한 시스템으로 풀어왔습니다.

- **타겟팅 서빙 아키텍처 설계 (O(logN), 0.11ms):** 광고 플랫폼에서 타겟팅 시스템을 처음부터 설계. batch 사전 연산 + 단순 PK 조회 구조로 AND/OR/NOT 조건 복잡도와 serving latency를 구조적으로 분리. 300만 유저 × segment N:N 매핑 약 2억 건을 복합 PK 단일 조회로 처리. 쿼리 평균 응답 0.11ms, 인프라 스택 MySQL 단일로 통일.
- **대용량 데이터 파이프라인 설계:** 일 2억 건 이상의 Audience 데이터를 처리하는 파이프라인 설계. 쿼리 로그 전수 분석으로 데이터 경량화(81%) 달성. 신규 타겟팅 차원(dimension)을 데이터팀 의존 없이 확장 가능한 Self-Serve 구조 확립. 31페이지 Runbook 작성으로 운영 안정화.
- **대규모 트래픽 처리 및 Fault Isolation:** DAU 100만, 피크 10M req/시간 처리 경험. SDUI 2-Phase 아키텍처 개편으로 ATF 렌더링 P90 670ms → 10ms (98% 단축). 광고 장애가 홈 화면 전체로 번지던 구조를 끊어 Fault Isolation 달성.
- **운영 자동화:** 파이프라인 모니터링 대시보드(원천→광고 송출 10단계 리니지 실측 감시)·배포 파이프라인 자동화(리드타임 3h → 1h, 실사용 23건) 직접 구축.

Work Experience

KCD 광고플랫폼팀 (2025.04 ~ 현재)

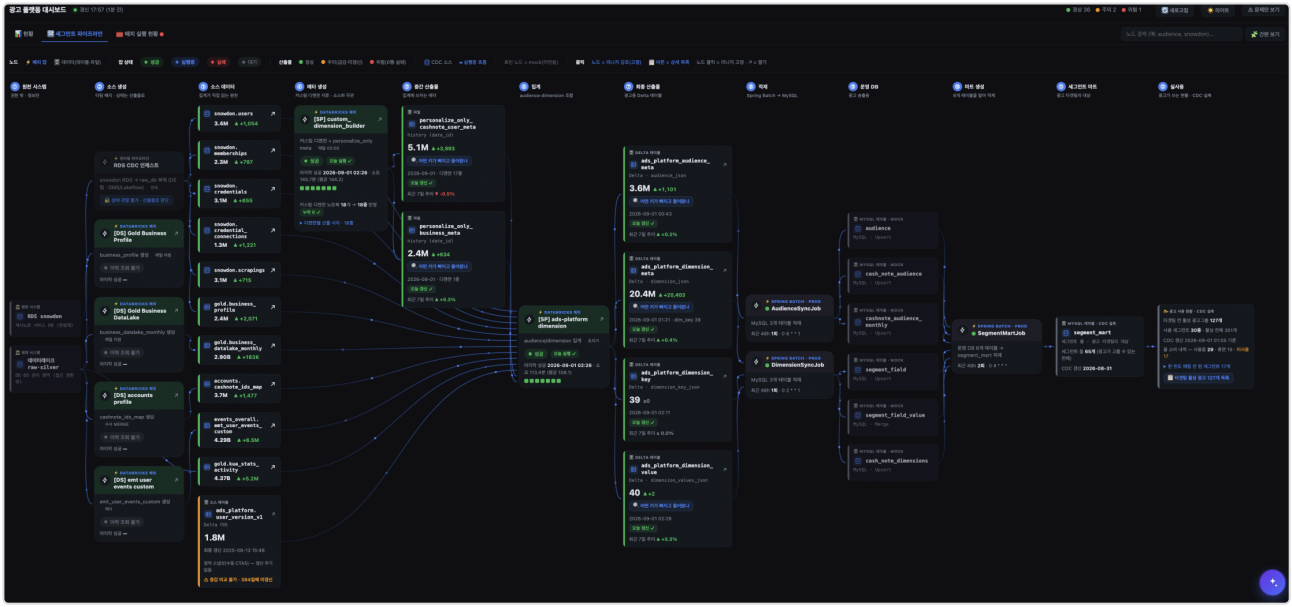
한국신용데이터 | 광고 플랫폼 백엔드 개발

- 신규 광고플랫폼 프로젝트 구축 — `admin, audience, serving, tracking-api` 도메인 초기 개발 진행

[아키텍처/성능] 타겟팅 서빙 배치 사전 연산 아키텍처 설계

신규 광고 플랫폼 설계 시 타겟팅 조건 평가를 **serving 시점에 수행할지, batch 시점에 사전 연산해둘지** 결정해야 했습니다. 실시간 연산 방식은 AND/OR/NOT 조건이 복잡해질수록 latency가 선형으로 증가하여, 타겟팅 로직의 복잡도와 serving 성능이 강하게 결합되는 구조적 문제가 있었습니다. 이 결합을 끊는 것이 설계의 목표였습니다.

- **결정의 본질:** "광고 송출 시점의 조건 검색"을 "batch 시점의 사전 연산 + 송출 시점의 복합 PK 조회"로 분리. NOT/IN/중첩 AND·OR 조건의 복잡도는 배치가 흡수하고, serving은 항상 단순 키 조회 하나로 응답.
- **대안 비교 후 선택:** 실시간 조건 검색 / 배치 사전 연산 두 방식을 비교하여, 타겟팅 조건 변경 빈도·워크로드 특성·운영 복잡도를 고려했을 때 배치 사전 연산 + MySQL 단일 스택이 가장 적합하다고 판단.
- **트레이드오프 명시:** segment 조건 변경 반영까지 최대 24h 지연. 광고 비즈니스는 **월 단위 사전 수주** 구조이므로 도메인 요구사항 안. 향후 실시간성 요구 발생 시 batch 주기 단축 / Redis 캐시 무효화 / Kafka incremental update 순으로 대응 가능하도록 회귀 옵션 설계.
- **결과:**
  - `segment_mart` 약 2억 건 규모를 복합 PK 단일 조회로 서빙. **쿼리 평균 응답 0.11ms** (DB 단 평균값 기준 — APM 미도입으로 p99 별도 측정은 불가).
  - 타겟팅 조건 복잡도(AND/OR/NOT/IN/중첩)와 serving latency 구조적 분리 달성.
  - 신규 플랫폼 인프라/운영 스택을 MySQL 단일로 통일. p99 tail latency의 외부 변동 요인을 구조적으로 제거.
- **[성능] 매핑 테이블 갱신 — 데드락 없는 무중단 데이터 로테이션**
  - `segment_mart` 매핑 테이블을 **1만 건 단위 Chunk Bulk Insert + created_at 기준 후행 삭제** 방식으로 갱신. 동일 segment에 대해 신규 데이터를 먼저 적재하고 과거를 지우는 순서로 서빙 단절·데드락 없이 갱신.
  - 생성 후 30일 이상 미사용 segment를 자동 비활성화(`active_status=false`)해 SegmentMartJob이 활성 segment만 처리하도록 범위 한정. segment 누적에 따른 배치 수행시간 선형 증가를 구조적으로 차단.
- **[설계] 타겟팅 전용 Self-Serve 차원 확장 파이프라인 신설**
  - 마케터 요청 기반 타겟팅 속성을 추가할 때마다 데이터팀 소관 user_meta·business_meta 수정이 필요했던 구조를 개선. 기존 정형 메타 파이프라인은 그대로 두고, 타겟팅 전용 `personalize_user_meta` · `business_meta` 를 생성하는 별도 파이프라인 신설. 지정 폴더에 `.py` 파일 추가 후 배치 실행만으로 dimension 정의 → admin field catalog 자동 갱신까지 자동 연결.
- **[운영] 파이프라인 모니터링 대시보드 구축:** 원천부터 광고 송출까지 10단계 리니지를 한 화면에서 실측 감시. 배치 성공 ≠ 데이터 신선도 사각지대 해소. 어제 대비 10% 이상 감소·3일 이상 미갱신·잡 실패·0행 발생 등 이상 조건을 비율·중앙값 기반으로 자동 감지 — 세그먼트 풀 34일 정체 등 기존에는 제보 전까지 알 수 없었던 이슈를 사전 포착 가능한 구조로.



[운영] 배포 파이프라인 자동화 (Deploy Pilot)

주 1회 prod 배포에서 GitHub · DevOps Portal · ArgoCD · Slack 사이의 대기가 반복됐습니다. **자동화 목표는 단계를 없애는 게 아니라 대기 구간에서 사람을 떼어내는 것.**

- **실행/UI 분리:** 브라우저 종료·세션 단절에도 배포가 멈추지 않도록 대시보드와 실행 프로세스를 분리. 비정상 종료 시 처음이 아닌 실패 단계부터 멱등 재개.
- **LLM 기반 CI 자동 수정:** 빌드 실패 감지 시 로그를 LLM에 전달해 원인 분석 → 코드 수정 → 재푸시를 에이전트 루프로 자동 반복 (최대 5회). 사람이 로그 확인·수정·재트리거하는 구간을 대체.
- **결과:** 배포 리드타임 3시간 → 1시간. 실사용 23건 (2026.08).

카카오스타일 (2022.06 ~ 2025.04)

지그재그(Zigzag) | 전시 도메인(홈·카테고리) 백엔드 개발

- 글로벌 패션 e-commerce 서비스 **지그재그 전시(홈·카테고리·상품리스트)** 백엔드 개발 담당
- Server Driven UI 기반으로 **대규모 트래픽(DAU 100만, 피크 10M/시간)** 처리 구조 설계 및 성능 개선

[성능/UX] Server Driven UI(SDUI) 호출 구조 개편 및 ATF 속도 98% 개선

- **아키텍처 고도화 (Single Query → Multi Query):** 화면 전체 데이터를 단일 쿼리로 조회하던 기존 방식의 병목(Head-of-Line Blocking)을 해소하기 위해, **Template(Skeleton)과 개별 컴포넌트 호출(Lazy Loading)로 분리된 2-Phase 아키텍처** 설계
- **로딩 속도 98% 단축:** 텍스트/이미지 로딩 전 스켈레톤 UI를 즉시 노출하는 전략을 통해, **ATF(Above The Fold) 렌더링 시간을 P90 기준 670ms → 10ms 내외로 단축**
- **안정성 확보 (Fault Isolation):** 컴포넌트 간 데이터 의존성을 제거하여, **특정 컴포넌트의 에러가 화면 전체 로딩 실패(Blank Page)로 전파되는 문제를 원천 차단**하고 부분 렌더링을 보장
- **확장성 개선:** 컴포넌트 추가 시 거대 단일 쿼리를 수정해야 했던 의존성을 끊어내어, 독립적인 테스트와 유연한 기능 확장이 가능한 구조 확립
- 사내 블로그 작성

[안정성/매출] 광고 및 홈 피드 시스템 리팩토링 (응답 속도 23% 개선)

- **아키텍처 디커플링:** 광고 서비스(AD)와 전시 서비스(UX)의 강결합을 끊고, UI 구성 책임을 UX로 이관하여 **광고 API 장애가 전체 화면 로딩 실패로 전파되는 문제(Fault Isolation) 해결**
- **응답 속도 23% 개선 (1.1s → 850ms):** 순차적(Sequential)으로 호출되던 외부 API 구조를 **Kotlin Coroutines**를 활용한 병렬(Non-blocking) 처리로 변경하고, **N+1 및 순환 참조 문제(api2 → ux → ad → ux)**를 해결하여 응답 속도 단축
- **유지보수성 확보:** 거대 `if-else` 레거시 코드를 **Factory/Strategy 패턴**으로 리팩토링하여 비즈니스 로직의 복잡도를 낮추고 신규 컴포넌트 추가 생산성 증대

🛠️ 주요 기술 스택

Language	Kotlin, Java
Framework	Spring Boot, Spring Batch, DGS (GraphQL), JPA
DB / Cache	MySQL, Redis, Elasticsearch, Ehcache, Caffeine
Infra	Docker, Datadog, Databricks, AWS S3, ArgoCD, GitHub Actions
Test	Kotest (BehaviorSpec), MockK
Architecture	Hexagonal Architecture, DDD

🔥 기타

- Kotlin Coroutines의 suspend function 활용 시 발생하는 컨텍스트 전파 및 트랜잭션 오류 트러블슈팅